Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 084 подписчика

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 5. Передача объектов в компонент

21 июня 2021

В этом уроке рассмотрим возможность передачи объектов в компонент

Для начала давайте вспомним, как взаимодействуют компонент и его модули.

В аннотации Component мы прописываем модули, которые компонент может использовать для создания объектов:

```
@Component(modules = [StorageModule::class, NetworkModule::class, MainModule::class])
interface AppComponent {
    // ...
}
```

Когда мы просим у компонента, например, объект DatabaseHelper, компонент берет объект StorageModule и вызывает у него соответствующий Provides метод, чтобы получить DatabaseHelper.

Это я к тому, что у компонента внутри есть объекты StorageModule, NetworkModule и MainModule. И он вызывает их методы, когда необходимо получить какой-либо объект.

Вопрос: откуда компонент берет эти объекты?

Давайте посмотрим на то, как мы создаем компонент:

```
class App: Application() {

    lateinit var appComponent: AppComponent

    override fun onCreate() {
        super.onCreate()
        appComponent = DaggerAppComponent.create()

    }
}
```

Мы вызываем только метод create. И совершенно точно мы не передаем компоненту никакие объекты модулей. Значит он создает их сам.

Если заглянуть в исходники метода create(), там мы увидим следующее:

```
return new Builder().build();
```

Используется простой билдер без каких-либо параметров.

Смотрим что делает метод build:

```
public AppComponent build() {
  if (storageModule == null) {
    this.storageModule = new StorageModule();
  }
  if (networkModule == null) {
    this.networkModule = new NetworkModule();
  }
  if (mainModule == null) {
    this.mainModule = new MainModule();
  }
  return new DaggerAppComponent(storageModule, networkModule, mainModule);
 }
}
```

Он создает объекты модулей, а затем передает их в конструктор компонента. А компонент уже вызывает Provides методы этих модулей, чтобы получать объекты.

Передача объекта

Все это отлично работает, когда у класса модуля пустой конструктор. Но бывают случаи, когда нам надо передать что-нибудь в модуль. Один из самых распространенных примеров - это передача Context.

Например, мы хотим, чтобы компонент мог давать нам такие объекты, как SharedPreferences или Resources. Для этого мы создаем модуль AppModule:

```
@Module
class AppModule {

    @Provides
    fun getPreferences(): SharedPreferences {
        return context.getSharedPreferences("prefs", MODE_PRIVATE)
    }

    @Provides
    fun getResources(): Resources {
        return context.resources
    }

}
```

Но есть проблема. Нам нужен Context. Даггер сам создать его не может. Да и мы не сможем написать такой Provides метод, который предоставит даггеру Context.

Для таких случаев есть возможность передать Context модулю в конструктор.

Перепишем конструктор AppModule, добавим в него Context:

```
@Module
class AppModule(private val context: Context) {

    @Provides
    fun getPreferences(): SharedPreferences {
        return context.getSharedPreferences("prefs", MODE_PRIVATE)
    }

    @Provides
    fun getResources(): Resources {
        return context.resources
    }

}
```

Теперь все в порядке с точки зрения модуля. У него есть context, и он сможет его использовать в Provides методах.

Добавляем AppModule в список модулей компонента:

```
@Component(modules = [AppModule::class, StorageModule::class, NetworkModule::class, MainModule::
interface AppComponent {
    // ...
}
```

Но у компонента возникает проблема. Его билдер теперь не может создать объект модуля AppModule, потому что конструктору AppModule требуется Context.

Это приводит к тому, что мы теперь не можем создать компонент простым вызовом метода DaggerAppComponent.create(). Нам надо будет использовать билдер напрямую. Нужно это для того, чтобы мы могли создать вручную объект модуля AppModule и передать его билдеру.

Заменяем вызов create на вызов билдера:

```
class App: Application() {

    lateinit var appComponent: AppComponent

    override fun onCreate() {
        super.onCreate()
        appComponent = DaggerAppComponent
            .builder()
            .appModule(AppModule(this))
            .build()
    }
}
```

От нас требуется передать билдеру объект только того модуля, который билдер не может создать сам - AppModule. Поэтому мы сами создаем этот модуль, используя App класс, как context, и передаем его билдеру.

Объекты трех остальных модулей билдер создаст без нашего участия, т.к. там простые конструкторы без аргументов.

Теперь все будет работать, и AppModule сможет использовать Context для создания объектов.

Граф

В примере выше мы передавали Context в модуль. И модуль использовал его для создания или получения других объектов. Но у даггера к нему доступа не было. Т.е. этот объект не был встроен в граф объектов компонента. Т.е. даггер не смог бы предоставить нам этот объект, если бы мы попросили. Или он не смог бы использовать Context при создании объектов вне модуля AppModule.

Чтобы это исправить, мы можем завернуть этот объект в отдельный Provides метод:

```
@Module
class AppModule(private val context: Context) {

    // ...

    @Provides
    fun getAppContext(): Context {
        return context
    }

}
```

Теперь и даггер сможет оперировать этим объектом.

Он сможет его использовать для создания других объектов, даже если они создаются в других модулях:

```
@Module
class StorageModule {

    @Provides
    fun provideDatabaseHelper(context: Context): DatabaseHelper {
        return DatabaseHelper(context)
    }

}
```

Даггер увидит, что в AppModule есть способ получения Context и использует его здесь для создания DatabaseHelper.

Назад Вперёд

Комментарии

Context — dev_course 18.11.2021 14:13
@Provides
fun getAppContext(): Context {
return context
}

выглядит как Bad practise. Подозреваю, что на контекст лучше вообще не держать ссылку, не то что передавать ее через граф. Хоть это и аппликӑйшн контекст.

Кстати, можно ли в модуль передать ActivityContext. Тоесть проинициализировать не весь граф модулей, но один из них доинициализировать на старте активити?

Ответить

RE: Context — dev_course 18.11.2021 19:03
Нашел ответ на вопрос, как передавать ActivityContext в Уроке8. Очень доходчиво, спасибо!

Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

3Comments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.